

Sarah-Maude Gagné
& Heidi Lavoie
Techniques de l'informatique
Groupe : 04318

Projet 3

Projet présenté à
M. Nicolas Payre
Département d'informatique
pour le cours
Exploration de nouvelles technologies

Cégep de Sherbrooke
8 décembre 2025

Table des matières

Partie 1	3
Informations projet	3
Équipe	3
Sujet choisi	3
Lien GitHub	3
Partie 2 - Exploration technique	4
Technologie(s) explorée(s)	4
Angular 18	4
Node.js.....	4
API Laravel.....	5
Principales découvertes	6
Clarifications.....	6
Ressources	6
Apport de l'AI dans le processus	7
Als utilisés	7
Réflexions critique	7
Sur les limites de la technologie	7
Sur le potentiel de la technologie	8

Partie 1

Informations projet

Équipe

Heidi Lavoie et Sarah-Maude Gagné

Sujet choisi

Angular 18 & API Laravel

Lien GitHub

https://github.com/twinkletoe103/projet3_nouvelles_technos.git

Générer un plan d'exploration (objectifs, livrables, échéancier).

Semaine 1 : Apprivoiser Angular 18, se familiariser avec l'environnement Angular et ses fichiers

Semaine 2 : Début du projet dans VSCode

Semaine 3 : Travailler sur le projet et préparation de la démonstration

Semaine 4 : Démonstration

Partie 2 - Exploration technique

Technologie(s) explorée(s)

Angular 18

Angular 18 est une version majeure du framework web open source de Google. Elle apporte des améliorations en performance, réactivité et outillage, tout en simplifiant le développement avec des nouveautés comme les composants autonomes (*standalone components*) et la possibilité de stocker la sortie d'un *pipe async* dans une variable avec la syntaxe *@let*.

Ce framework est utilisé pour créer des applications web modernes et performantes, comme des SPA (*Single Page Applications*). Il facilite la gestion d'état grâce aux nouveaux signaux réactifs et offre une meilleure composabilité pour les interfaces complexes. Ses utilisations typiques incluent des applications nécessitant une interface utilisateur réactive, un rendu rapide et une bonne scalabilité, comme des tableaux de bord, des plateformes d'e-commerce ou des applications internes. Les développeurs l'exploitent pour optimiser le chargement des pages, améliorer la composabilité et simplifier le débogage.

Les meilleures pratiques incluent l'utilisation des nouvelles fonctionnalités comme le Control Flow et les composants autonomes, l'adoption de la gestion d'état avec les signaux et l'optimisation des performances grâce au *lazy loading*. De plus, il est important de maintenir une structure de projet propre et de sécuriser les applications contre les failles.

Angular 18 s'intègre facilement avec de nombreux outils et environnements modernes. Il fonctionne bien avec des backends comme Node.js, Laravel ou Firebase grâce à ses services HTTP, et s'appuie sur des outils de développement tels que VS Code, Git ou Docker.

Node.js

Node.js est un environnement d'exécution JavaScript open-source qui permet d'écrire des programmes côté serveur en JavaScript. Il utilise le moteur V8 de Google et une architecture événementielle, non bloquante et asynchrone pour créer des applications réseau performantes et évolutives, comme des serveurs web ou des applications en temps réel.

C'est couramment utilisé pour le développement backend d'applications web, la création d'APIs (REST et autres), la construction d'applications temps réel (comme les jeux et les plateformes de chat), le streaming de données et la création de microservices. Son architecture le rend particulièrement efficace pour les opérations d'entrée/sortie (I/O) intensives.

Les meilleures pratiques pour Node.js incluent l'externalisation de la configuration avec des variables d'environnement, la gestion des erreurs avec des bibliothèques comme Winston ou Pino, la structuration du code selon des modèles comme le MVC, et l'utilisation de frameworks comme Express pour une gestion efficace des requêtes. Il est également crucial d'éviter de bloquer la boucle

d'événements en utilisant correctement les opérations asynchrones, les promesses et `async/await`, et de tester rigoureusement le code avec des outils comme Jest ou Mocha.

API Laravel

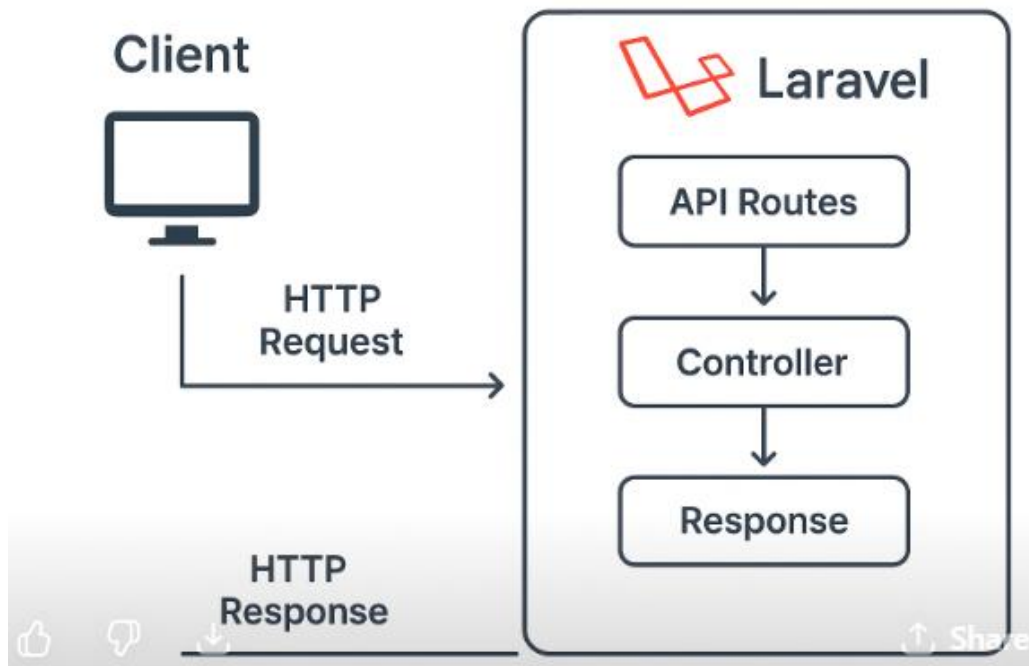
Une API Laravel est une interface permettant à des applications externes d'interagir avec une application Laravel via HTTP, généralement en utilisant JSON comme format de données. Laravel fournit des outils pour créer facilement des routes, contrôleurs et ressources API. Une API sépare le back-end (Laravel) du front-end.

Laravel offre des fonctionnalités robustes pour créer des API, un système de routage flexible, un ORM pour interagir avec les bases de données, des outils de validation, un système d'authentification et d'autorisation, et la gestion de la sérialisation des réponses avec les ressources API. Ces fonctionnalités permettent de construire des API sécurisées, évolutives et maintenables.

Les meilleures pratiques d'un API Laravel est d'utiliser JSON comme format de réponse, de séparer les routes web et les routes API. Il faut aussi gérer les erreurs avec des réponses standardisées (`response()->json(..., 400)`). L'utilisation de tokens ou de Auth, documenter l'API et limiter les requêtes avec *throttle* pour éviter des brèches de sécurités.

Les utilisations courantes d'un API Laravel sont : le back-end pour des applications mobiles ou SPA, le back-end d'applications qui exposent des données publiques ou des micro-services et intégration entre différents systèmes.

Diagramme du fonctionnement d'un API Laravel, généré par Chat GPT.



Principales découvertes

1. Angular 18 est un framework web open source complet créé par Google.
2. C'est basé sur TypeScript pour ajouter de la structure et de la sécurité.
3. Architecture basée sur les composants, donc chaque partie est modulaire et réutilisable.
4. Avant Node.js, JavaScript était presque exclusivement côté client.
5. Node.js inclut npm.
6. Node.js utilise un système de modules pour structurer le code.

Clarifications

Composants autonomes : Composants qui n'ont pas besoin d'être déclarés dans un module pour fonctionner.

Sortie d'un *pipe async* : Valeur finale d'une donnée asynchrone.

SPA : Une application web qui charge une seule page HTML et met à jour son contenu dynamiquement sans recharger toute la page à chaque action. (Single Page Application)

Signaux réactifs : Valeurs qui "réagissent" automatiquement aux changements.

Composabilité : Capacité à être construit à partir de composants indépendants et réutilisables.

Control Flow : Permet de contrôler l'affichage ou le comportement des éléments dans le template en fonction de conditions ou de boucles. (Flux de contrôle)

Lazy loading : Charger une partie de l'application que lorsque c'est nécessaire, au lieu de tout charger au démarrage.

Open-source : Code source disposition au public.

Npm : Gestionnaire de paquets le plus populaire pour JavaScript.

ORM : Convertir des données entre une base de données relationnelle (tables) et des objets dans un langage de programmation orienté objet. (Object-Relational Mapping)

Throttle : Limiter la fréquence ou la vitesse à laquelle une opération peut être exécutée.

Ressources

Cours de 3 jours, pour développeurs Web qui veulent apprendre Angular avec de bonnes bases :

https://www.globalknowledge.com/us-en/course/154895/introduction-to-angular-essentials/?utm_source=google&utm_medium=cpc&utm_campaign=GK_NA_Google_NonBrand_DSA&utm_content={adgroup}&utm_term=angular%2018%20training&campaignid=21758139996&adid=765158792151&gad_source=1&gad_campaignid=21758139996&gbraid=0AAAAAD_i1UYOqQfHyZ8iyTB_r5pbQFGfPw&gclid=Cj0KCQiAieblBhDmARIsAE8PGNIFVkJZN8LF5-v4_mALHUKgSRdQd3Ju53owr1zp5ZZH1yFXHJsz3EYUaAphqEALw_wcB

Documentation de Angular 18 :

<https://angular.dev/overview>

Documentation Node.js :

<https://nodejs.org/docs/latest/api/>

Documentation Laravel :

<https://laravel.com/docs/12.x/eloquent-resources>

Documentation API Laravel :

<https://api.laravel.com/docs/12.x/index.html>

Apport de l'AI dans le processus

Recherche de ressources d'apprentissages sur les technologies et clarifications de concepts plus compliqués.

Als utilisés : Chat GPT, Claude et Copilot.

Réflexions critique

Sur les limites de la technologie

Angular 18

1. Assez complexe, il y a beaucoup de concepts à maîtriser.
2. Les bundles sont volumineux donc le chargement est plus lent pour les petites applications.
3. L'architecture est moins flexible pour « bricoler » comme elle est plus stricte.
4. Les migrations sont lourdes, donc les mises à jour peuvent demander d'importants ajustements.

Node.js

1. Comme c'est « single-thread », des calculs peuvent bloquer l'application.
2. L'asynchrone est complexe, le code pourrait être difficile à maintenir.
3. Certains packages sont obsolètes ou non-sécurisés.
4. Comme il n'y a pas de typage natif, il y a des erreurs possibles.

API Laravel

1. Laravel est plus lourd que d'autres frameworks, il peut ralentir l'API.
2. Les plus gros projets sont très gourmands en mémoire.
3. Peut être complexe pour des débutants à cause de l'écosystème (middleware, resources, services...)

Sur le potentiel de la technologie

Angular 18

1. Code sûr et maintenable.
2. Technologie très réactive et dynamique.

Node.js

1. Idéal pour une application en temps réel.
2. Capable de gérer de nombreuses connexions simultanées.
3. Comme npm est présent, il y a beaucoup de package disponible, ce qui accélère le développement.

API Laravel

1. Intégration facile d'un API
2. Très sécuritaire, protections intégrées contre CRSF, XSS, injections SQL...
3. Facilité à connecter à différents front-end ou d'autres services.